

AI MOOD UPLIFT APP

Complete Technical & File-by-File Documentation

1. Introduction

The AI Mood Uplift App is an intelligent, therapeutic music application designed to analyze a user's emotional state via text and voice, guiding them through intermediate emotional states using dynamic, AI-curated playlists based on the 'Emotional Ladder' concept.

2. Project Overview

The project operates on a Flutter Frontend (Dart) and a Flask Backend (Python).

- **User Input:** Users provide feedback via text or speech.
- **AI Analysis:** Backend utilizes Google Gemini AI to analyze text, determine mood, and intensity.
- **Music Delivery:** Backend queries Spotify APIs; Frontend streams via YouTube.
- **Personalization:** App learns user preferences, un-logging helpful songs and blocking unhelpful ones.

3. Architecture and Workflow

1. App Start: Flutter initializes state. Splash screen checks SharedPreferences auth tokens.
2. Input: Users use native microphone (speech_to_text) or type.
3. Analysis: Frontend sends text to Flask. Backend uses Gemini NLP and queries Spotify.
4. Playback: Global audio player streams tracks. Interstitial feedback sheets appear.

AI Mood Uplift App - Technical Documentation

4. File-Wise Documentation

1. Backend: app.py

- **A. File Name:** app.py
- **B. Role:** Central Python backend server running Flask. Handles REST APIs, Gemini AI logic, and Spotify integrations.
- **C. Main Components:** Flask initialization, SQLAlchemy config, API Routes.
- **D. Logic Description:** register()/login(): Auth endpoints. process_mood(): Gemini NLP analysis determining trajectory. get_song(): Spotify fetching with BlockedList/SongPlayLog filtering. log_session_outcome(): Saves session metrics. get_ai_insights(): Gemini behavioral summary generation.
- **E. Data Transport:** Accepts HTTP JSON POSTs/GETs. Returns structured JSON (moods, tokens, tracklists).
- **F. Architecture Conn:** Imports models.py. Consumed exclusively by api_service.dart.
- **G. Contribution:** The computational brain bridging UI with NLP and music databases.

2. Backend: models.py

- **A. File Name:** models.py
- **B. Role:** Defines SQL database schema and ORM mappings using SQLAlchemy.
- **C. Main Components:** User, Session, SongPlayed, MoodTransition, Preference, SongPlayLog, MoodSessionLog classes.
- **D. Logic Description:** User: Authentication details. Session/MoodTransition: Timeline tracking. Preference: Global song blocks/likes. SongPlayLog: Granular played/skipped telemetry. MoodSessionLog: Session summaries for AI.
- **E. Data Transport:** Maps Python objects into SQLite database rows.
- **F. Architecture Conn:** Imported solely by app.py.
- **G. Contribution:** Provides robust persistent storage enabling AI learning and history tracking.

3. Frontend: main.dart

- **A. File Name:** main.dart
- **B. Role:** Entry point for Flutter app, initializing environment, theme, and global providers.
- **C. Main Components:** main(), MoodUpliftRoot.
- **D. Logic Description:** main(): Binds Flutter, instantiates AppState, calls loadFromPrefs(). MoodUpliftRoot: Wraps app in ChangeNotifierProvider and uses MaterialApp.builder to globally overlay the PlayerScreen.

AI Mood Uplift App - Technical Documentation

- **E. Data Transport:** Injects AppState into the widget tree.
- **F. Architecture Conn:** Initializes app_state.dart. Routes to splash or login.
- **G. Contribution:** Foundational wrapper ensuring state and media overlays persist across routes.

4. Frontend: app_state.dart

- **A. File Name:** app_state.dart
- **B. Role:** Central state management controller (Provider). Handles business logic, audio state, and API sync.
- **C. Main Components:** AppState (extends ChangeNotifier).
- **D. Logic Description:** playAudio()/nextTrack(): just_audio playback controls; auto-logs skipped songs. setApiResult(): Processes mood transitions and triggers _evaluateSongs(). _evaluateSongs(): Blocks unhelpful songs, un-logs uplifting ones. loadFromPrefs(): Loads auth tokens.
- **E. Data Transport:** Emits notifyListeners(). Sends/receives data from ApiService.
- **F. Architecture Conn:** Consumed by all widgets. Relies on api_service.dart.
- **G. Contribution:** The absolute core tying UI, media playback, and backend synchronization replacing local logic.

5. Frontend: api_service.dart

- **A. File Name:** api_service.dart
- **B. Role:** Networking abstraction layer handling HTTP calls to Flask.
- **C. Main Components:** ApiService (static class).
- **D. Logic Description:** processMood(): POSTs text for AI analysis. getNextSong(): Fetches tracks for target mood. logSong()/requestUnlogSongs(): Sends telemetry data. getAilInsights(): Fetches Gemini summaries.
- **E. Data Transport:** Parses Dart Maps to JSON payloads. Returns decoded JSON to AppState.
- **F. Architecture Conn:** Called by AppState and processing screens. Connects to app.py.
- **G. Contribution:** Isolates HTTP complexity/timeouts, keeping UI code clean.

6. Frontend: mooduplift_splash.dart

- **A. File Name:** mooduplift_splash.dart
- **B. Role:** Initial startup screen rendering animations and executing auth checks.
- **C. Main Components:** SplashScreen, _checkAuth().
- **D. Logic Description:** initState(): Starts visual pulse animation. _checkAuth(): Delays 2.2s, checks AppState.isLoggedIn, routes to MoodInputScreen or LoginScreen.

AI Mood Uplift App - Technical Documentation

- **E. Data Transport:** Reads boolean from AppState.
- **F. Architecture Conn:** Routes to login or home screens.
- **G. Contribution:** Secures entry point and masks async loading latency behind aesthetics.

7. Frontend: mooduplift_login.dart

- **A. File Name:** mooduplift_login.dart
- **B. Role:** Authenticates returning users.
- **C. Main Components:** LoginScreen, _handleLogin().
- **D. Logic Description:** _handleLogin(): Validates TextFields, calls ApiService.login(), updates AppState with JWT, routes to home. Shows SnackBars on error.
- **E. Data Transport:** Sends email/password strings.
- **F. Architecture Conn:** Connects to api_service.dart and mood_input.dart.
- **G. Contribution:** Enforces user identity isolation for accurate histories.

8. Frontend: mooduplift_create_account.dart

- **A. File Name:** mooduplift_create_account.dart
- **B. Role:** Registers new users and captures baseline demographics.
- **C. Main Components:** CreateAccountScreen, _handleRegister().
- **D. Logic Description:** Dropdown validation: Ensures valid age/gender constraints. _handleRegister(): Submits demographics to ApiService.register(), auto-logs in on success.
- **E. Data Transport:** Sends raw demographic strings via ApiService.
- **F. Architecture Conn:** Sibling to login; routes to mood_input.dart.
- **G. Contribution:** Bootstraps personalized AI capabilities by securing initial database user rows.

9. Frontend: mood_input.dart

- **A. File Name:** mood_input.dart
- **B. Role:** Primary interface for users to express emotional state via typing/speech.
- **C. Main Components:** MoodInputScreen, _toggleMic(), _chipsRow(), _analyze().
- **D. Logic Description:** _toggleMic(): Uses speech_to_text for dictation, throttling state updates. _chipsRow(): Pre-fills text area with templates. _analyze(): Validates input, saves to AppState, routes to processing screen.
- **E. Data Transport:** Writes transcribed text strings to AppState.
- **F. Architecture Conn:** Routes to mooduplift_processing.dart.
- **G. Contribution:** Crucial sensory ingestion point generating data for NLP models.

AI Mood Uplift App - Technical Documentation

10. Frontend: mooduplift_processing.dart

- **A. File Name:** mooduplift_processing.dart
- **B. Role:** Loading interface masking high-latency Gemini API calls.
- **C. Main Components:** ProcessingScreen, _startProcessing().
- **D. Logic Description:** _startProcessing(): Calls ApiService.processMood(). If AI determines emotional jump is large, routes to next_target.dart. Otherwise, injects data to AppState.setApiResult and starts playback directly.
- **E. Data Transport:** Retrieves inputText; parses complex dict response from app.py.
- **F. Architecture Conn:** Bridge between input and next_target / player screens.
- **G. Contribution:** Handles complex architectural decision trees for psychological progression paths.

11. Frontend: next_target.dart

- **A. File Name:** next_target.dart
- **B. Role:** Enforces 'Emotional Ladder' by presenting realistic intermediate target moods.
- **C. Main Components:** NextTargetScreen, _buildOption(), _startJourney().
- **D. Logic Description:** _buildOption(): Dynamically renders selectable mood stepping stones. _startJourney(): Commits target to AppState, calls getNextSong(), and pops screen to reveal PlayerScreen.
- **E. Data Transport:** Reads trajectory parameters populated during processing.
- **F. Architecture Conn:** Pops cleanly to expose main.dart global overlay.
- **G. Contribution:** Validates the core psychological therapeutic architecture by preventing dangerous emotional leaps.

12. Frontend: player_and_feedback.dart

- **A. File Name:** player_and_feedback.dart
- **B. Role:** Persistent global audio player interface overlaying the entire app.
- **C. Main Components:** PlayerScreen, _buildRecord(), _buildControls().
- **D. Logic Description:** _buildRecord(): Visual spinning vinyl UI. _buildControls(): Maps Play/Pause/Skip buttons to AppState functions. Listens to just_audio progress streams.
- **E. Data Transport:** Reads currentTrackIndex, playbackProgress, isPlaying from AppState.
- **F. Architecture Conn:** Globally injected by main.dart. Triggers FeedbackSheet on intervals.
- **G. Contribution:** Standard media consumption UI driving the constant therapeutic session.

13. Frontend: feedback_sheet.dart

AI Mood Uplift App - Technical Documentation

- **A. File Name:** feedback_sheet.dart
- **B. Role:** Modal pop-up capturing real-time mood shifts during track playback.
- **C. Main Components:** FeedbackSheet, _submit().
- **D. Logic Description:** Dual Sliders/Emojis: Captures exact numeric intensity and classified mood.
_submit(): Calls AppState.setApiResponse(isUpdate=true). AppState then evaluates if recent songs were uplifting (un-log) or unhelpful (block).
- **E. Data Transport:** Dispatches updated currentMood and intensity params to AppState.
- **F. Architecture Conn:** Invoked over PlayerScreen. Drives AppState telemetry.
- **G. Contribution:** The critical measurement tool closing the AI feedback loop and driving machine learning.

14. Frontend: mooduplift_history.dart

- **A. File Name:** mooduplift_history.dart
- **B. Role:** Visualizes chronological session histories and AI-generated text insights.
- **C. Main Components:** HistoryScreen, _loadHistory().
- **D. Logic Description:** _loadHistory(): Pulls local tracking from AppState.loadHistory(). Asynchronously spawns ApiService.getAiInsights() to fetch Gemini summaries without blocking UI.
Layout Builders: Draw visual timeline vectors.
- **E. Data Transport:** Parses dates, intensities, emojis. Receives lists of string insights.
- **F. Architecture Conn:** Hosts AiInsightsCard. Connects to api_service.dart.
- **G. Contribution:** Provides user validation and transparent reporting of their long-term therapeutic progress.

15. Frontend: ai_insights_card.dart

- **A. File Name:** ai_insights_card.dart
- **B. Role:** Aesthetic UI component specifically rendering NLP AI responses.
- **C. Main Components:** AiInsightsCard.
- **D. Logic Description:** build(): Transforms raw text array strings into beautifully styled layout blocks mimicking conversational intelligence.
- **E. Data Transport:** Accepts List<String> parameters.
- **F. Architecture Conn:** Nested natively inside HistoryScreen.
- **G. Contribution:** Maximizes UI presentation quality for dry NLP algorithmic text generation.

16. Frontend: settings.dart

- **A. File Name:** settings.dart
- **B. Role:** Account administration configuration interface.

AI Mood Uplift App - Technical Documentation

- **C. Main Components:** SettingsScreen, _handleLogout(), _clearHistory().
- **D. Logic Description:** _handleLogout(): Nullifies local SharedPreferences and logs out. _clearHistory(): Purges user's SongPlayLog data permanently in the DB via API.
- **E. Data Transport:** Calls void functions.
- **F. Architecture Conn:** Connects to AppState logouts and ApiService destructive endpoints.
- **G. Contribution:** Ensures absolute user autonomy over their generated session data.

17. Frontend: exit_confirmation.dart

- **A. File Name:** exit_confirmation.dart
- **B. Role:** Dialog intercepting native OS back commands to prevent accidental session termination.
- **C. Main Components:** ExitConfirmationDialog.
- **D. Logic Description:** onWillPop override: Intercepts gestures. If confirmed, gracefully runs AppState._finalizeLiveSession() before yielding.
- **E. Data Transport:** Returns booleans to Navigator.
- **F. Architecture Conn:** Wrapped globally around active session screens.
- **G. Contribution:** Maintains database integrity guaranteeing duration metrics are logged correctly on close.

18. Frontend: main_secondary.dart

- **A. File Name:** main_secondary.dart
- **B. Role:** Auxiliary application entry point generally utilized for specific testing or mock environments.
- **C. Main Components:** Mock configurations.
- **D. Logic Description:** Mirrors main.dart structure but injects detached parameters cleanly.
- **E. Data Transport:** Standard initialization data.
- **F. Architecture Conn:** Parallel to main.dart.
- **G. Contribution:** Provides a clean sandbox execution pathway for developers without corrupting production states.

5. End-to-End Execution

1. Launch: main.dart injects AppState. mooduplift_splash.dart verifies auth dynamically.
2. Intake: mood_input.dart parses device microphones. Text is submitted via api_service.dart.
3. Processing: app.py Gemini analysis returns nested playlist dicts with ladder mapping logic.
4. Evaluation: player_and_feedback.dart streams YouTube payloads. feedback_sheet.dart intercepts metrics. AppState filters models.py BlockedLists accordingly.